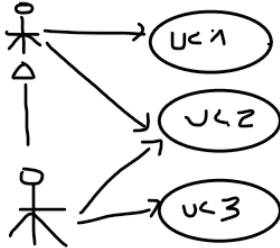


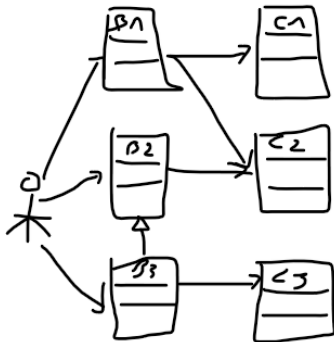
Lezione 8 13/11/2023 GOF

venerdì 17 novembre 2023 14:27

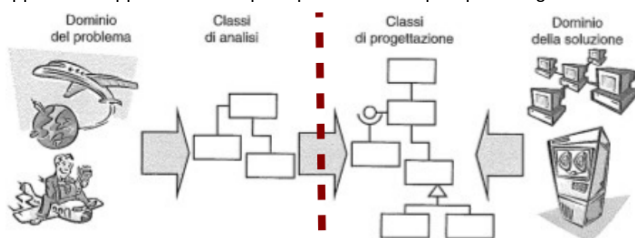
Quindi riepilogando il BCE : **le boundary non devono assolutamente scambiarsi entity con i controller** . Al massimo nel metodo delle boundary verranno invocate un'operazione del controller. Vediamo un esempio :



Attenzione però al contesto : in questo caso si parla di generalizzazione tra attori , il che non è sbagliato ma dipende dal contesto . Applicandogli il BCE si ha che :

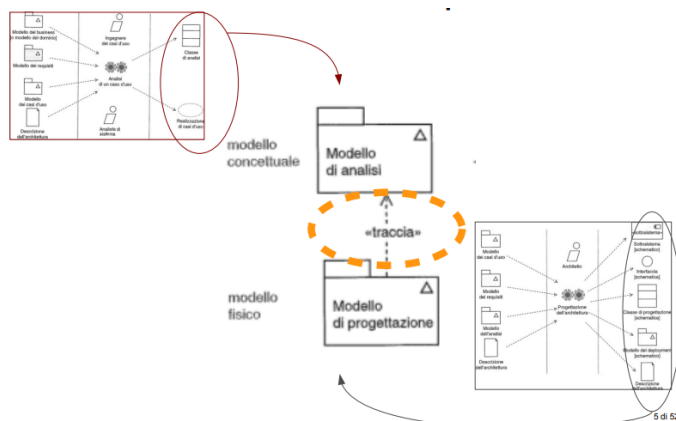


Ma attenzione alla generalizzazione tra Boundary : dal punto di vista concettuale è corretta , ma dal punto di vista concettuale è alquanto difficile da implementare (non è sbagliato) . Cominciamo a parlare de GOF, ma prima facciamo un breve richiamo su UML e sul contesto generale : ricordiamo che UML è un linguaggio per visualizzare, specificare, costruire e documentare i sistemi software , quindi la non imposizione di un determinato processo di sviluppo e non applicazione dei principi OO. Mentre per quanto riguarda il contesto generale :



- obiettivo dell'analisi è capire il problema e sviluppare modelli (visuali) che descrivono ciò che si vuole ottenere indipendentemente dalle tecnologie e da aspetti implementativi
- l'analisi si focalizza sul raffinamento dei requisiti (funzionali) in concetti rappresentabili come elementi software

- obiettivo delle fasi di progettazione è raffinare i modelli di analisi con l'intento di includere dettagli e scelte che più si prestino alle tecnologie di sviluppo ed ai contesti di deployment scelti
- I modelli di progettazione raffinano aspetti tecnologici rispetto ai modelli di analisi, ma non si focalizzano su dettagli implementativi, che sono invece delegati alle fasi più orientate alla scrittura del codice



Notiamo che il BCE fa parte della fase di analisi , ma si sposta in realtà verso la fase di progettazione .

Ora però sorge un problema : come si deve/dovrebbe progettare un sistema ? Come possiamo valutare idee/soluzioni più adatte al contesto?? Non si sa , ma facciamo la seguente ipotesi : dato un problema , abbiamo un metodo per risolverlo ed inoltre abbiamo anche la seguente tesi : questo metodo aumenti la comunicazione tra progettisti : per esempio tecniche sistematiche di supporto alla progettazione . Andiamo ora nel dettaglio : vediamo i **design pattern** : ovvero una

descrizione di un problema ricorrente + schema di soluzione : come per esempio BCE, polimorfismo, metamorfosi, legge di Demetra. Quindi per design pattern si intende una regola che deve essere rispettata per arrivare ad una soluzione : è caratterizzato da un nome , una soluzione che può essere istanziata in diversi contesti ed una discussione su conseguenze e variazioni che ne derivano . Questi pattern sono raggruppati in base a categorie e problemi . In dettaglio :

- Nome pattern e Classificazione - dovrebbero essere altamente significativi - Intento - breve descrizione per mostrare ciò che fa il pattern - qual è il suo fondamento/intento? - Sinonimi - altre nomenclature non ufficiali usate per riferenziare il pattern - Motivazione - scenario che illustra un problema di progettazione e il modo in cui la struttura di classi e oggetti definita nel pattern lo risolve - Applicabilità - situazioni dove il pattern può essere utilizzato	- Struttura - rappresentazione grafica del pattern - Partecipanti - classi e oggetti che fanno parte del design e le loro responsabilità - Collaborazioni - come collaborano i partecipanti per potersi assumere le loro responsabilità - Conseguenze - come fa il pattern a raggiungere i propri obiettivi - pro e contro nell'applicazione del pattern	- Implementazione - suggerimenti e tecniche per implementare il pattern - problemi specifici connessi a un particolare linguaggio di programmazione - Codice di esempio - frammenti di codice (le descrizioni originali sono per C++ o Smalltalk) - Usi conosciuti - utilizzo di pattern in sistemi reali - Pattern correlati - altri pattern che sono in relazione
---	--	---

In generale sono classificati così :

- Scopo : definisce il dominio di applicazione : risolve responsabilità di creazione oggetti (Creator)
 - Creational : creator
 - Behavioral : comportamento
 - Structure : struttura
- Contesto : a quali classi / oggetti si applica ?
 - Class: relazioni tra classi e sotto-classi
 - Object : tipi di dato o istanze a run-time
- Riassumendo :

		Purpose		
		Creational	Structural	Behavioral
a.	Class	Factory Method (107)	Adapter (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Flyweight (195) Observer (293) State (305) Strategy (315) Visitor (331)

- Possibili combinarli tra loro :

Creational && Class: delegano parte del processo di creazione di un oggetto a sottoclassi

Creational && Object: delegano parte del processo di creazione di un oggetto ad altri oggetti

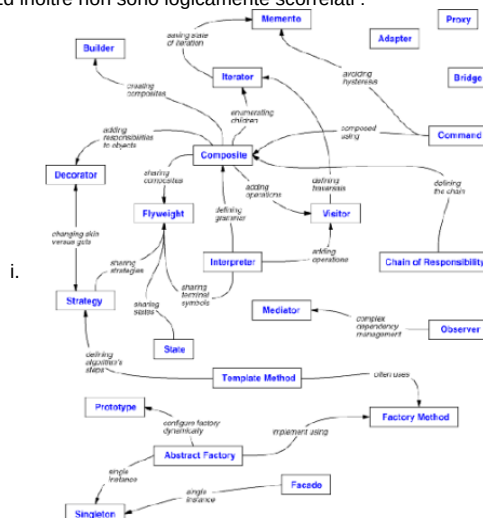
Structural && Class: utilizzano l'ereditarietà per comporre classi,

- Structural && Object**: descrivono modi per raggruppare oggetti

Behavioral && Class: utilizzano ereditarietà per descrivere algoritmi e flusso di controllo

Behavioral && Object: descrivono come gruppi di oggetti cooperano per eseguire un compito che un singolo oggetto non potrebbe portare a termine da solo

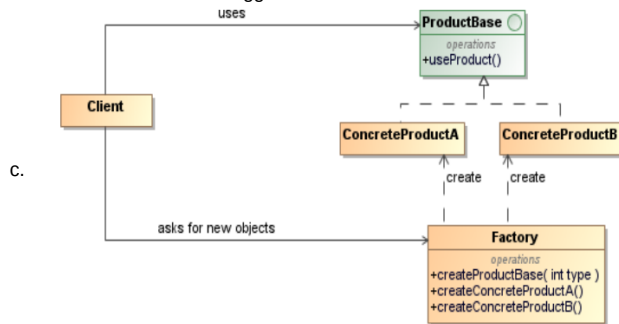
- Ed inoltre non sono logicamente scorrelati :



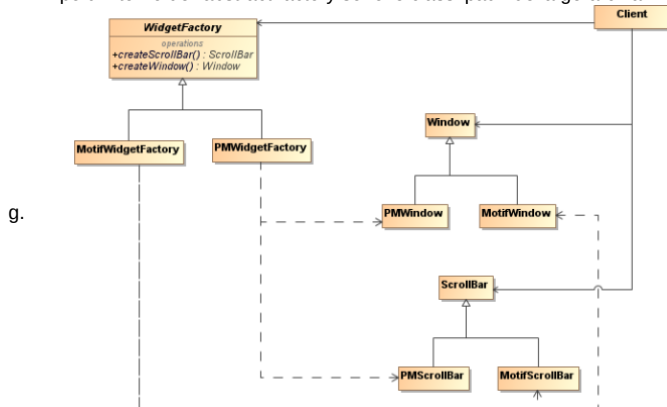
Vediamo quelli **creazionali (creator)** : forniscono astrazione dal processo di istanziazione degli oggetti e rendono il sistema indipendente dalla modalità di creazione , sono basati sul concetto di ereditarietà per scegliere la classe da istanziare , la quale istanziazione è delegata ad altro oggetto . Questo tipo di pattern è caratterizzato da una maggiore flessibilità, in quanto si

ha a che fare solo con le interfacce degli oggetti definite mediante classe astratte . Sono factory , abstract factory e singleton :

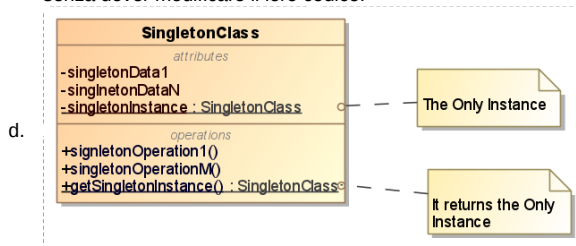
1. **Factory** : permette la creazione di oggetti simili a run-time
 - a. Chiamato anche virtual constructor
 - b. Si deve determinare l'oggetto corretto solo a run-time



- d. Il client usa tipi di dato che assomigliano all'interfaccia
- e. Factory è parte di soluzione polimorfa e funge da Creator per tutti i tipi di dato del dominio
- f. **Regola pratica : vietato fare la new dovunque , ma viene fatta solo a run time dalla factory.**
- g. **Aggiunta nuovi prodotti molto facile**
2. **Abstract factory** : permette la creazione di famiglie di oggetti correlati e/o dipendenti
 - a. Chiamato anche kit
 - b. Esistono diverse famiglie di prodotti, ognuna delle quali deve essere gestita in modo opportuno.
 - c. Isola classi concrete, consente di cambiare la famiglia di prodotti utilizzata , promuove coerenza utilizzo prodotti , **aggiunta di nuove tipologie di oggetti è difficile.**
 - d. **Da notare che il client non ha la responsabilità di creare unico oggetto, ma due**
 - e. **La WidgetFactory è abstract, mentre le classi che hanno la responsabilità di creare le istanze degli oggetti sono le concrete factory (MotifWidgetFactory e PMWidgetFactory).**
 - f. Tipo di ritorno dell'abstract factory sono le classi padri della gerarchia .



3. Singleton : assicura che una classe abbia una sola istanza in tutta l'applicazione , fornendone un punto globale a tale istanza
 - a. Unica classe con responsabilità di creare altri classe
 - b. Deve esistere unicamente un'istanza di tale classe e deve essere accessibile ai client attraverso un unico punto di accesso condiviso da tutti .
 - c. Questa istanza deve essere potere estesa a sottoclassi (cambiando il costruttore da private a protected) , e i client devono essere in grado di usare le istanze estese senza dover modificare il loro codice.



- e. Caratterizzata da 3 elementi :
 - i. **Attributo privato statico del tipo della classe**
 - ii. **Operazione che restituisce unica istanza : se istanza è null (da inizializzazione) la crea e la ritorna:**
 1. **Public static Singleton getInstance()**
 2. **{**
 - a. **If (instance==null)**
 - b. **{**
 - i. **Instance=new SingletonInstance();**
 - c. **}**
 - d. **Return instance;**
 3. **}**
 - iii. **Costruttore privato**

